

Containers in the Cloud

From packaged apps to managed clusters with Kubernetes and Google Kubernetes Engine (GKE)

Instructor-led module · approx. 60 min + quiz

What you will be able to do



Explain containers

Describe what a container packages and why it is more portable than a VM



Name the core K8s objects

Tell apart Pods, Deployments and Services, and the job each one does



Run core kubectl workflows

Deploy, expose, scale and roll out an app, both imperatively and declaratively



Choose a GKE mode

Decide between Autopilot and Standard based on how much control you need

Roadmap for this module

01

Containers

What they are, how they differ from VMs, how they scale

02

Kubernetes

Cluster anatomy, Pods, Deployments, Services, rollouts

03

GKE

Managed Kubernetes, Autopilot vs Standard, cluster features

?

Quiz

8 questions with answer reveals and discussion

PART 01

Containers

Package once, run the same way on a laptop, in test and in production

Why containers exist



The problem

- Code depends on libraries, runtimes and OS settings
- Each environment is slightly different
- Result: an app that works in dev but breaks in production

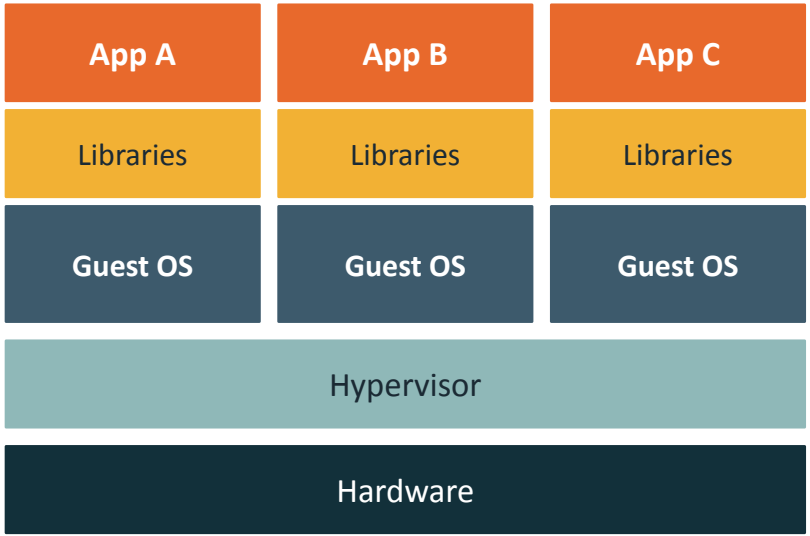


The shipping container idea

- Standard box: the crane does not care what is inside
- Software containers bundle code plus everything it needs
- Any host with a container runtime can run the box unchanged

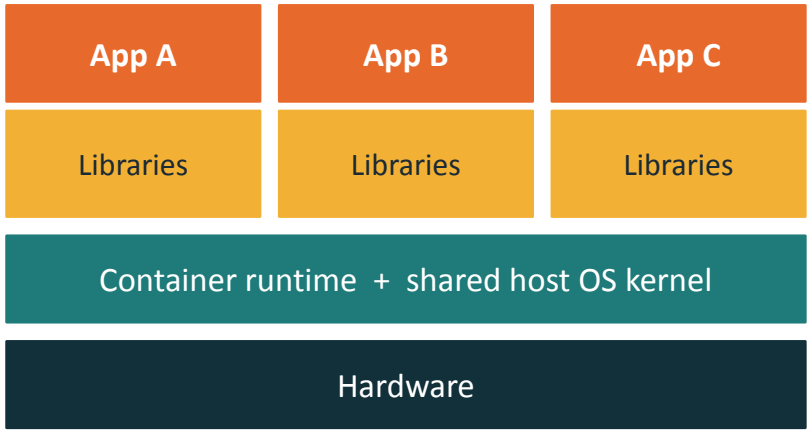
Virtual machines vs containers

Virtual machines



Each VM carries a full OS: heavier, slower to boot

Containers



Containers share the kernel: light, start like a process

Where containers sit on Google Cloud

More control

More automation



IAAS

Compute Engine

You manage VMs, OS patches and scaling

CONTAINERS

GKE

You manage app containers; Kubernetes handles placement and healing

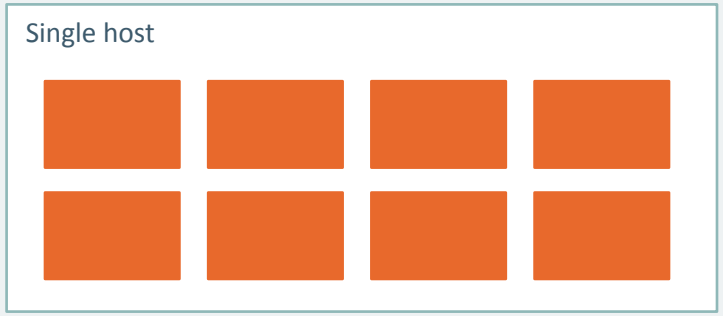
PAAS

App Engine

You bring code; Google runs everything underneath

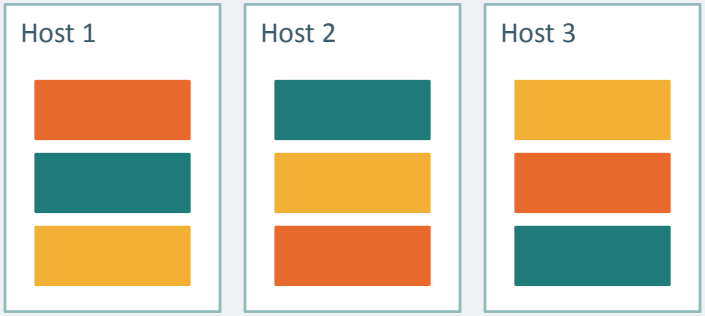
Two ways to scale with containers

Copy one container



The whole app runs in one image; add identical copies in seconds to meet demand.

Split into microservices



Each service (UI, cart, payments) is its own container and scales independently across hosts.

PART 02

Kubernetes

An open-source orchestrator that keeps many containers on many machines in the state you asked for

Kubernetes works from desired state



1. Declare

You describe what you want: which image, how many copies, how to expose it



2. Observe

Kubernetes continuously compares what is running with what you declared

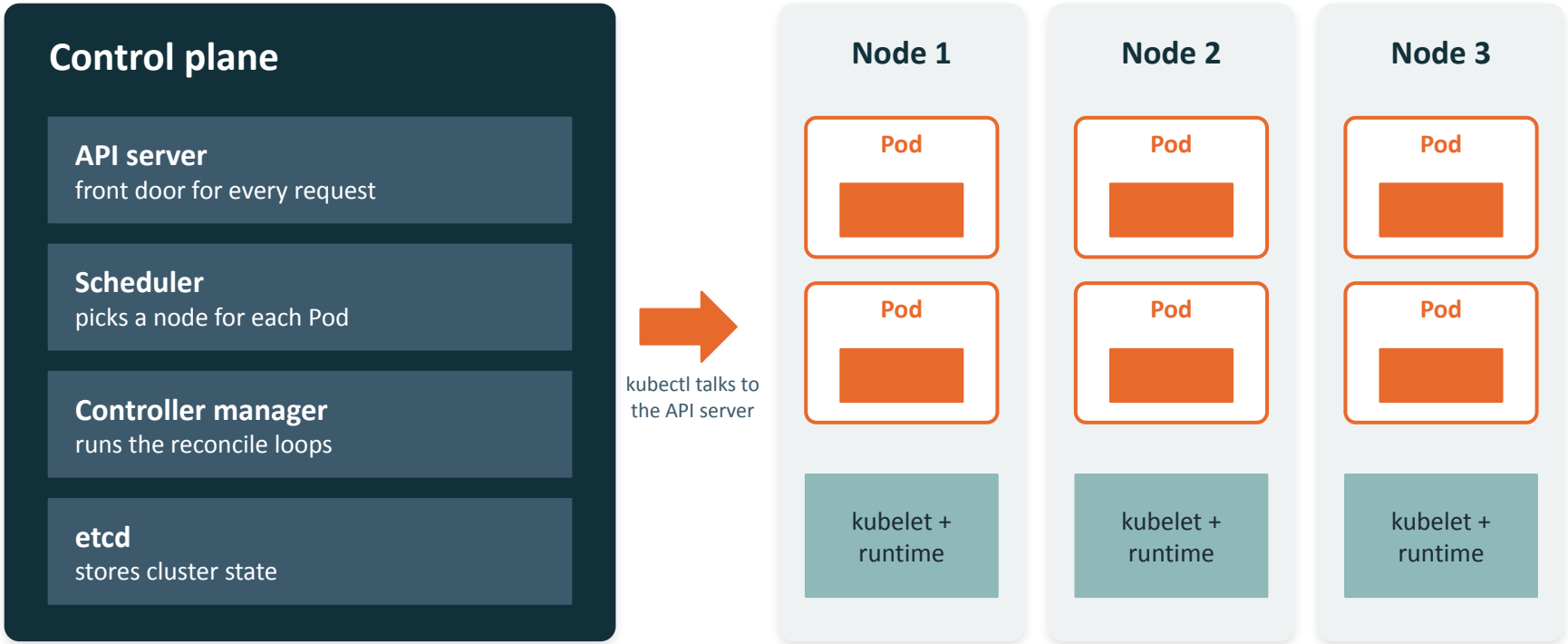


3. Reconcile

It starts, stops or moves containers to close any gap - automatically

The loop never stops: if a node fails, step 2 notices and step 3 replaces the lost containers.

Anatomy of a cluster



Three objects you use every day



Pod

Smallest deployable unit

Wraps one container (sometimes a few tightly coupled ones) that share an IP, ports and storage volumes.



Deployment

Keeps N copies running

Manages identical replicas of a Pod, replaces failed ones, and handles rollouts and rollbacks.

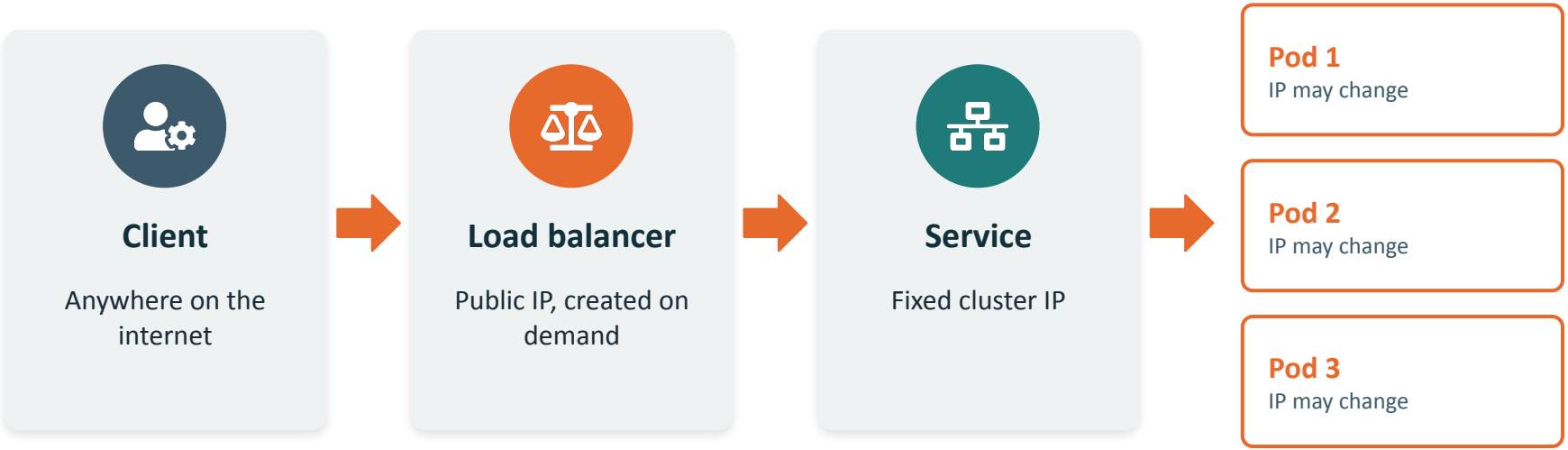


Service

Stable address for Pods

Groups matching Pods behind one fixed IP and load-balances traffic across them.

How traffic reaches your Pods




```
$ kubectl expose deployment storefront --port=80 --type=LoadBalancer
```

Imperative vs declarative

Imperative: tell it each step

```
$ kubectl create deployment storefront \
    --image=nginx:1.27
$ kubectl scale deployment storefront \
    --replicas=3
$ kubectl expose deployment storefront \
    --port=80 --type=LoadBalancer
```

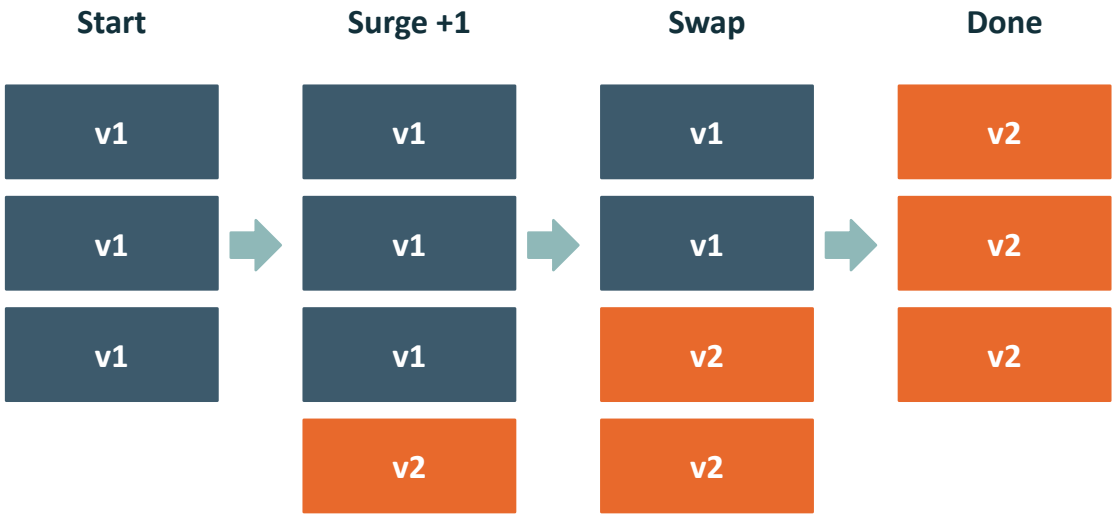
Great for learning and quick tests. Hard to review, repeat or version.

Declarative: describe the end state

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: storefront
spec:
  replicas: 3
  selector:
    matchLabels: {app: storefront}
  template:
    metadata:
      labels: {app: storefront}
    spec:
      containers:
      - name: web
        image: nginx:1.27
        ports: [{containerPort: 80}]
```

```
$ kubectl apply -f storefront.yaml
```

Rolling updates without downtime




```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

maxSurge: 1 - at most one extra Pod above the desired count

maxUnavailable: 0 - never drop below full capacity

Trigger with kubectl rollout / kubectl set image, or edit the manifest and kubectl apply

kubectl quick reference

Command	What it does
`kubectl get pods`	List running Pods
`kubectl get deployments`	Check desired vs ready replicas
`kubectl get services`	Find a Service's cluster and external IP
`kubectl expose deployment ...`	Create a Service in front of a Deployment
`kubectl scale ... --replicas=N`	Change the number of replicas
`kubectl apply -f file.yaml`	Create or update objects from a manifest
`kubectl rollout status | undo`	Watch or roll back an update

PART 03

Google Kubernetes Engine

Kubernetes as a managed service: Google runs the control plane so you can focus on workloads

What GKE takes off your plate



Self-managed Kubernetes

- Provision and secure control plane machines
- Back up and restore etcd
- Plan and run every version upgrade
- Wire up load balancers, logging, monitoring



With GKE

- Google hosts and operates the control plane
- You get one endpoint for all Kubernetes API calls
- Nodes are Compute Engine VMs grouped into a cluster
- Built-in integration with Google Cloud services

Two modes: Autopilot and Standard



Autopilot

Recommended

- Google manages nodes, scaling and upgrades
- Baseline security and networking set for you
- Optimised for production; pay for what Pods use
- Less to configure, less to get wrong



Standard

Full control

- You configure and manage the nodes
- You choose machine types and node pools
- You own cluster tuning and optimisation
- Pick it when you need that specific control

Cluster features you get with GKE



Load balancing

Google Cloud load balancing for your Services



Node pools

Group nodes with different machine types in one cluster



Autoscaling

Grow or shrink the node count with demand



Auto-upgrades

Keep node software current automatically



Auto-repair

Detect and fix unhealthy nodes



Observability

Logging and monitoring via Google Cloud Observability

End-to-end: from zero to a running app

1. **Create an Autopilot cluster**

```
gcloud container clusters create-auto k1 --region=us-central1
```

2. **Deploy the app**

```
kubectl create deployment storefront --image=nginx:1.27
```

3. **Expose it publicly**

```
kubectl expose deployment storefront --port=80 --type=LoadBalancer
```

4. **Scale out**

```
kubectl scale deployment storefront --replicas=5
```

5. **Roll out a new version**

```
kubectl set image deployment/storefront nginx=nginx:1.27.1
```

KEY TAKEAWAYS

Three things to remember

Containers

bundle code and dependencies and share the host kernel, so they are light and portable

Kubernetes

reconciles reality with your declared state using Pods, Deployments and Services

GKE

runs Kubernetes for you; start with Autopilot unless you need Standard's control

KNOWLEDGE CHECK

Module quiz

8 questions · each question is followed by an answer slide with an explanation

Question 1 of 8



Which statement best describes a container?

A A virtual machine with its own guest operating system

B An isolated package of code and dependencies that shares the host OS kernel

C A physical server dedicated to one application

D A managed database service

Answer 1 of 8

Which statement best describes a container?

A	A virtual machine with its own guest operating system
B	**An isolated package of code and dependencies that shares the host OS kernel** ✓
C	A physical server dedicated to one application
D	A managed database service

ANSWER: B

Containers virtualise the operating system, not the hardware. They bundle the app and its dependencies but share the host kernel, so they are lightweight and start like a process.

Question 2 of 8



What is the smallest unit you can create and deploy in Kubernetes?

A A container

B A Pod

C A node

D A Deployment

Answer 2 of 8

What is the smallest unit you can create and deploy in Kubernetes?

- **A** A container
- **B** **A Pod** ✓
- **C** A node
- **D** A Deployment

ANSWER: B

Kubernetes never schedules a bare container. It deploys Pods, which wrap one or more containers that share networking and storage.

Question 3 of 8



Pod IP addresses change as Pods are recreated. Which object gives clients a stable endpoint?

A ReplicaSet

B Volume

C Service

D Namespace

Answer 3 of 8

Pod IP addresses change as Pods are recreated. Which object gives clients a stable endpoint?

- **A** ReplicaSet
- **B** Volume
- **C** **Service** ✓
- **D** Namespace

ANSWER: C

A Service defines a logical set of Pods and a policy for reaching them, behind one fixed IP. Clients call the Service, not individual Pods.

Question 4 of 8



Where do the machines that make up a GKE cluster's nodes come from?

A App Engine instances

B Cloud Storage buckets

C Bare-metal servers you supply

D Compute Engine virtual machines

Answer 4 of 8

Where do the machines that make up a GKE cluster's nodes come from?

A	App Engine instances
B	Cloud Storage buckets
C	Bare-metal servers you supply
D	**Compute Engine virtual machines** ✓

ANSWER: D

GKE nodes are Compute Engine instances grouped together to form a cluster. This is why GKE can use Compute Engine features such as load balancing.

Question 5 of 8



You want to change a Deployment from 3 to 5 replicas the declarative way. What do you do?

A Run kubectl expose with a new port

B Edit replicas in the YAML file and run kubectl apply -f

C Run kubectl get pods five times

D Delete and recreate the cluster

Answer 5 of 8

You want to change a Deployment from 3 to 5 replicas the declarative way. What do you do?

A	Run kubectl expose with a new port
B	**Edit replicas in the YAML file and run kubectl apply -f** ✓
C	Run kubectl get pods five times
D	Delete and recreate the cluster

ANSWER: B

Declarative management means updating the desired state in the configuration file and applying it. Kubernetes then works out how to add the two extra Pods.

Question 6 of 8



A rolling update uses maxSurge: 1 and maxUnavailable: 0. What happens?

A All old Pods are deleted, then new ones start

B One new Pod is added before each old Pod is removed, so capacity never drops

C Only one Pod is ever updated

D The update pauses until an admin approves each Pod

Answer 6 of 8

A rolling update uses maxSurge: 1 and maxUnavailable: 0. What happens?

A All old Pods are deleted, then new ones start

B **One new Pod is added before each old Pod is removed, so capacity never drops** ✓

C Only one Pod is ever updated

D The update pauses until an admin approves each Pod

ANSWER: B

maxSurge 1 allows one extra Pod above the desired count; maxUnavailable 0 forbids going below it. So Kubernetes adds a new Pod, waits until it is ready, then retires an old one.

Question 7 of 8

A small team with no dedicated ops staff wants Google to manage nodes, scaling and security baselines. Which GKE mode fits?

A Autopilot

B Standard

C Self-managed Kubernetes on VMs

D Any mode - they are identical

Answer 7 of 8

A small team with no dedicated ops staff wants Google to manage nodes, scaling and security baselines. Which GKE mode fits?

A **Autopilot** ✓

B Standard

C Self-managed Kubernetes on VMs

D Any mode - they are identical

ANSWER: A

In Autopilot, GKE manages node configuration, autoscaling, auto-upgrades and baseline security and networking. It is Google's recommended mode unless you need Standard's control.

Question 8 of 8



Which of these is NOT a cluster management feature provided by GKE?

A Node auto-repair

B Automatic upgrades of node software

C Automatically rewriting your application source code

D Node pools

Answer 8 of 8

Which of these is NOT a cluster management feature provided by GKE?

- **A** Node auto-repair
- **B** Automatic upgrades of node software
- **C** **Automatically rewriting your application source code** ✓
- **D** Node pools

ANSWER: C

GKE manages cluster infrastructure - repair, upgrades, node pools, scaling, load balancing and observability. Your application code is always your responsibility.



Questions and discussion

Next step: hands-on lab - create a GKE Autopilot cluster, deploy, expose, scale and roll out an update